Use LLMs to Build Repair Tools, Not Patch Generation: Cost-Effective Automated Program Repair

Anonymous submission

Abstract

Large language models (LLMs) repair buggy submissions to programming assignments well, but they are deployed *one query per submission*: every buggy program triggers a fresh model call, so cost, latency, and external dependence scale with enrollment and recur every term, while the specification-level reasoning being purchased is identical across a class. Caching and cheaper models shrink only the constant, not the structure. We propose CoEP, which moves the LLM from the marginal-cost to the fixed-cost column: *spend the model once per assignment, not once per submission*. An LLM agent reads the assignment and a clustered sample of worked (buggy $\rightarrow$ fixed) pairs and authors the assignment-specific layer of a local repair tool: AST rewrite rules named after the misconceptions they fix, atop a shared symbolic substrate of generic edits and reference-guided minimal-hunk transplant from the class's own correct solutions. Every candidate patch is validated against the instructor's tests before it is kept; the tool can never self-report success, and submissions it abstains on escalate to a conventional per-submission call, so coverage is never sacrificed. On the standard benchmark of this literature, the synthesized tool repairs *more* submissions with *smaller* patches than the strongest hand-built symbolic tool, on that tool's own corpus and metric, with the entire model spend collapsed into a one-off synthesis plus a small escalation tail, so everything repaired locally costs no model call at all, offline and deterministically. A price-invariant break-even analysis shows when amortization wins and why cheaper models and caching do not change the answer, and the artifact yields what a per-query API cannot: reproducible minimal patches and a per-class misconception profile.

Introduction

Timely, individualized feedback is the scarcest resource in programming education: introductory courses enroll hundreds of students on campus and tens of thousands online, each producing buggy submissions no instructor can examine one by one. Automated program repair (APR) has therefore long been proposed as a feedback engine for programming assignments: given a buggy submission and the assignment's test suite, produce a small patch that makes it correct while preserving the student's own approach (Yi et al. 2017; Gulwani, Radicek, and Zuleger 2018; Wang, Singh, and Su 2018; Hu et al. 2019; Ahmed et al. 2022).

Three generations of systems have pursued this goal. Early systems established the setting: test-driven repair whose patch must stay close to the student's program, because an unrelated rewrite passes the tests but teaches nothing (Yi et al. 2017). A second generation mined the class's own solution space, matching each buggy program against previously accepted solutions (Gulwani, Radicek, and Zuleger 2018; Wang, Singh, and Su 2018; Hu et al. 2019; Ahmed et al. 2022). This repairs cheaply, but it requires a large solution pool and fails whenever a submission matches no reference (Orvalho, Janota, and Manquinho 2025). LLM-based repair removed both constraints and now fixes most real student programs at modest per-bug prices (Zhang et al. 2024, 2022; Xia and Zhang 2024; Liu et al. 2024).

What the third generation fixed in coverage, it broke in cost structure. The deployment default is one model query per buggy submission, so spending grows with enrollment and restarts every term. The queries themselves are massively redundant. A single assignment is attempted by hundreds of students whose failures cluster into a few recurring misconceptions (Gulwani, Radicek, and Zuleger 2018; Hu et al. 2019), and on the corpus we study, about two thirds of a repair prompt is byte-identical across submissions to the same assignment. Paying a model to rediscover the same off-by-one for the three-hundredth time is not intelligence at work; it is a billing artifact. The standard remedies do not change this shape. Cheaper models and provider caching shrink the constant (Anthropic 2025; OpenAI 2024), and retrieval enriches each query (Dai et al. 2026), but every submission still triggers a model round trip with its marginal cost, latency, and data egress. The question is not how much a call costs, but where in the pipeline the LLM is invoked.

Our answer is to spend the LLM once per assignment, not once per submission. Agent research has shown that LLMs can author reusable code and amortize it across many uses (Cai et al. 2024; Wang et al. 2024), and classical synthesis has learned per-assignment fix rules from examples (Rolim et al. 2017). CoEP (Cost-Effective Program repair) instantiates this idea for assignment repair. An LLM agent reads the assignment once and authors its problem-specific layer: AST rewrite rules named after the misconceptions they fix.

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

These rules run on a shared symbolic substrate, a generic edit library plus reference-guided transplant from the class's correct pool. Every candidate patch must pass the instructor's tests before it is kept, so a tool can never self-report success. The frozen tool then repairs submissions locally in about a second at zero marginal LLM cost, and the few it abstains on escalate to a conventional per-submission call.

The resulting artifact beats the hand-built state of the art on its own benchmark and metric: 98.48% repair rate at 0.289 mean relative patch size, against Refactory's 90.8% at 0.40, over 1,686 real student programs. We are explicit about attribution. The rule layer produces about 13% of local fixes, at near-minimal patch size and with a misconception label each; the transplant engine produces the rest, and an ablation shows the symbolic substrate alone already carries the coverage. The claim is therefore not that an LLM wrote a state-of-the-art repairer. The claim is the invocation structure: one cheap build plus a small fallback tail replaces an inference per submission, and the artifact it feeds exceeds the hand-built tool.

The stakes scale with deployment. In one course the per-submission bill is dollars; on an online judge it is not. The public Codeforces dump our benchmark draws from aggregates 12.6M submissions to over 10,000 problems (Penedo et al. 2025), At the standard \$3/\$15 per Mtok rate of the models we run, our measured per-submission repair costs about \$125 per thousand submissions, re-paid as every new submission arrives. The comparison is also robust to pricing. If a build costs C_{build} and locally resolves a fraction $(1-f)$ of submissions that would each have cost Δ, amortization wins past $N^* = C_{\text{build}}/(\Delta(1-f))$, and N^* is invariant to token price when build and query spending follow the same schedule. The artifact also offers what per-submission inference cannot at any price. Every patch is re-executed against the instructor's tests before it is emitted, so it can never be invalid or unverified code, and the rule layer edits by AST-located source-span splice rather than re-rendering the module. It is deterministic and runs offline, so student code never leaves the institution. Its rule-fired repairs aggregate into a per-class misconception profile that no API call returns.

This paper makes the following contributions:

- **A cost-structure reformulation of assignment repair.** We move the LLM from per-submission marginal cost to per-assignment fixed cost, with a price-invariant break-even analysis $N^* = C_{\text{build}}/(\Delta(1-f))$: amortization is a structural advantage, not a constant-factor one that cheaper models erase.
- **The CoEP system.** A builder agent synthesizes misconception-named AST rewrite rules through a generate–validate–refine loop (Madaan et al. 2023; Shinn et al. 2023), atop a shared substrate of generic edits and reference-guided minimal-hunk transplant. Every patch is independently test-validated, and abstentions escalate to a strategy-matched LLM fallback.
- **An empirical result on the baseline's own corpus and metric.** CoEP's local repair exceeds Refactory on both coverage and patch size at negligible one-off build cost, with an honest mechanism attribution, a framework-only ablation isolating the LLM layer's contribution, and per-builder analyses across two commodity models.
- **Artifacts.** We release the harness, dataset-import pipeline, prompts, synthesized repairers, and all raw model outputs.[1]

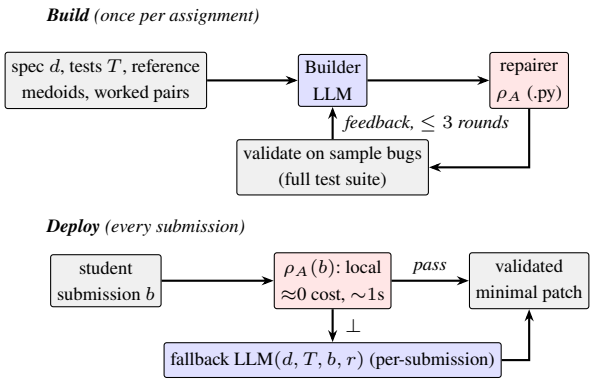


Figure 1: CoEP. LLM effort (blue) is spent once per assignment to synthesize a local repairer (red); at deployment, the large majority of submissions never touch a model.

The Cost Anatomy of Per-Submission Repair

Decomposing the per-submission prompt of our PyDex reproduction shows where the money goes (Zhang et al. 2024; Xia and Zhang 2024). Averaged over the Refactory corpus, about two thirds of a query's input tokens are byte-identical across every submission to the same assignment: the instructions, the visible tests, and, in the zero-shot layout, the description. Only the buggy program and the retrieved exemplars vary. Provider caching cannot reach that redundancy, because it discounts a leading shared *prefix* rather than shared content. PyDex's layouts place the variable material first, so the longest identical prefix across submissions is about ten tokens, a few percent of the prompt. The remaining reuse is within a submission, across the K sampled candidates, and even that is gated by cacheable minimums of 1,024 to 4,096 tokens that these 1,300-token prompts meet only at the smaller end.

Two consequences follow. First, caching is a discount, not a fix, and here not even a discount: the redundancy is real, but provider caching serves it at the wrong granularity. Second, the redundancy is exactly the amortizable part. What is constant per assignment, namely understanding the specification, the tests, and the space of likely mistakes, can be reified once into an executable artifact. That is the move CoEP makes, and the rest of the paper quantifies what it buys.

Approach

Problem Formulation

An *assignment* is a triple $A = (d, T, R)$: a natural-language description d, a test suite $T = \{(x_i, y_i)\}$ (function-entry or judge-style stdin/stdout), and the pool R of correct student

[1] Anonymized repository submitted as supplementary material.

solutions, which the setting assumes is available at feedback time, as every per-assignment system from CLARA to Refactory does (Gulwani, Radicek, and Zuleger 2018; Hu et al. 2019). A *repair* of a buggy submission b is a program p that passes all of T. Patch quality is measured, never gated: RPS, the tree-edit distance between the ASTs of b and p normalized by the size of b, following Refactory's definition (Hu et al. 2019), is reported for every repair, since a small patch is feedback on the *student's* program while a rewrite passes tests but teaches nothing (Yi et al. 2017).

Conventional LLM-based APR maps each b to a query $\mathrm{LLM}(d, T, b)$. CoEP instead synthesizes, once per assignment, a *repairer* $\rho_A : b \mapsto p \cup \{\bot\}$, an ordinary module with no model access, and deploys the hybrid: $\mathrm{repair}(b) = \rho_A(b)$ if $\rho_A(b) \neq \bot$ (local, $\approx$0 marginal cost), else $\mathrm{LLM}(d, T, b, r)$: the conventional call, strengthened with one reference r chosen to match the student's strategy by clustering R on program structure and drawing r from the cluster nearest b.

Repairer Synthesis (once per assignment)

Build-set selection (local, no LLM cost). The builder does not see the whole pool; it sees a *chosen* sample. We cluster the buggy pool on a structural view (TF-IDF over normalized-AST paths, identifiers folded) and, walking clusters largest-first, take from each the member with the smallest tree-edit distance to its nearest correct solution, a clean single-misconception exemplar rather than the medoid's typical but tangled program, paired with that nearest correct program. The result is a set of worked (buggy $\rightarrow$ fixed) pairs: frequent misconceptions first, each shown *with* its minimal fix, so a weak builder needs only to generalize a transformation it is shown, not infer it.

Rule authoring. The builder LLM receives d, T, reference solutions (structural-cluster medoids of R), and the worked pairs, and emits the assignment-specific rule module: AST rewrite rules, each named after the misconception it targets and tagged with the failure symptoms it addresses. The contract is strict. A rule locates its edit on the AST and splices it as a source-span replacement, never re-rendering the module, which would reformat untouched lines. It emits candidates rather than claimed fixes, and it never rewrites a whole function: a rule that cannot express a targeted edit must abstain so the submission escalates.

Generate–validate–refine. A single-shot module can contain its own defects, so the harness closes the loop: it loads the module, runs it on the sample bugs, *independently* validates every candidate patch by executing the full suite (the repairer's own claims are never trusted), and, while any sample remains unrepaired, returns the module with per-bug outcomes and the still-failing sources for revision, up to $R_{\max}$=3 builder calls. The best-validated round is kept, and build cost is exactly the token count of these exchanges, measured from the raw strings.

Local Repair (every submission)

The frozen tool repairs in three stages, cheapest and smallest-patch first, validating every candidate against the full suite before it is kept:

(1) Named rules + generic search. Rules are *candidate generators*, composed by a symptom-ordered combination search: the submission's failing-test signature ranks rules whose declared symptoms match, then single rules, then pairs, under an explicit candidate budget. A shared generic library of comparison flips, off-by-one range and index shifts, and Boolean-operator flips is always on, so common slips are covered regardless of what the builder wrote; it is the inversion of the classical mutation-operator space (Just et al. 2014; Kim et al. 2013; Liu et al. 2019). Every patch that survives carries the ordered list of rule names that produced it, provenance that aggregates into a per-class *misconception profile*.

(2) Reference-guided minimal-hunk transplant. Most student bugs need a structural edit no token-level rule expresses; the correct pool supplies it. We diff b against its structurally nearest correct solutions, rename the reference's variables to the student's (parameters by position, locals by first-appearance order), and adopt the *smallest* set of differing line-hunks that passes the tests, candidates ordered smallest-first and batch-validated, returning the minimum-RPS passer. This is reference reuse in the lineage of CLARA, Sarfgen, and Refactory (Gulwani, Radicek, and Zuleger 2018; Wang, Singh, and Su 2018; Hu et al. 2019), at hunk granularity rather than block granularity, which is what keeps patches smaller than the hand-built tool's.

(3) Escalation. A submission both stages abstain on takes the fallback $\mathrm{LLM}(d, T, b, r)$, its cost counted against CoEP in every comparison. Abstention is not the only trigger. Local repair's acceptance test is the instructor's suite, which says nothing about patch size, so transplant can return a patch larger than the student's whole program while the rule layer must abstain whenever it cannot express a targeted edit. We give transplant the same contract by escalating any passing local patch whose RPS exceeds 1: a patch at least as large as the program it repairs is a rewrite, not a repair, under any assignment's standards. The smaller of the local and model patches is kept, so escalating can cost a call but never coverage or patch size.

The threshold is a constant, not a calibrated quantity, and that is a finding rather than a simplification. We swept it over 38 settings — relative rules anchored on the median or mean with MAD, standard-deviation or interquartile spread, bare quantiles, multiples of the assignment's median, and fixed constants — and every one of them leaves repair rate unchanged and every one of them beats all three baselines on repair rate, patch size, turnaround and dollars simultaneously (RQ5). Within that family, making the threshold relative to the assignment does not pay: a single corpus-wide cutoff sits on the same cost-quality frontier or better, because RPS is already normalized by program size and a second per-assignment normalization spends calls where the tool is doing well rather than where it is doing badly. Since no setting is measurably best, we take the only one whose value is a definition rather than a fitted constant, and report the rest as a sensitivity band.

(4) Self-learning. Each successful fallback fix can be generalized by the LLM into a new named rule, kept only if the augmented tool then repairs that miss on its own and

rolled back otherwise. Absorption writes to a staging copy the evaluated pass never consults, so no submission's outcome depends on which submissions preceded it and both deployment configurations remain reproducible from the frozen artifact alone. The accumulated rules are a separate artifact, evaluated by a second frozen pass and priced to themselves rather than to the configuration they were gathered alongside (Section). Self-learning spends model calls only on submissions that already escalated, so it too leaves the build claim untouched.

The local path runs on a CPU in $\sim$1–2 s per submission, deterministically and offline; a defective tool degrades to $\perp$ (escalation), never to a wrong "repair", because the grading harness re-validates every patch before a student sees it.

Experiments

Research Questions. The primary research questions (RQs) addressed by our evaluation are as follows:

- **RQ1** (*repair effectiveness*): Assignment by assignment, how many submissions does CoEP repair and how much of each program does it edit, against a state-of-the-art symbolic tool and a per-submission LLM repairer?
- **RQ2** (*cost effectiveness*): How much does the once-per-assignment invocation structure reduce turnaround time and dollars, past what submission volume does the one-off build pay off, and can provider prompt caching or cheaper model tiers close the gap?
- **RQ3** (*where the repairs come from*): How do the LLM-authored rules, the shared symbolic substrate, the reference transplant, and the fallback each contribute to the repairs and to the misconception labels the tool reports?
- **RQ4** (*learning from its own escalations*): Do the rules CoEP induces from its own fallback fixes improve the frozen tool on submissions they were not induced from?
- **RQ5** (*what the conclusion survives*): Does the advantage hold across assignment difficulty, from CS1 to Codeforces problems rated 800 to 2000, across both LLM providers, and across the choice of escalation threshold?

Baselines. **Refactory** (Hu et al. 2019) is the classical symbolic per-assignment tool: it refactors the class's correct solutions to match each buggy program's structure and transplants the divergent blocks. On its own corpus we use its published results; on Codeforces we run the released tool in its strongest configuration. **PyDex** (Zhang et al. 2024) is per-submission LLM repair: each buggy submission is sent to the model in its few-shot layout, which prepends the three most similar (buggy, correct) exemplars retrieved from the class pool, sampled candidates are filtered by the test oracle, and the surviving candidate closest to the student's program is returned. **PyDex$_{\mathbf{PC}}$** re-prices the same queries under provider prompt caching, as an upper bound that assumes a cache hit wherever the provider's rules allow one. **CoEP** is ours: a repairer synthesized once per assignment and model, evaluated in two deployment configurations plus one that measures what a term of use adds. **CoEP$_{\mathbf{L}}$** runs the frozen local tool alone, with zero LLM use at deployment, and accepts any patch its test oracle passes. That is deliberately the same acceptance rule Refactory uses, so the comparison between them is about repair, not about a size filter one of them applies. **CoEP$_{\mathbf{LF}}$** adds the per-submission fallback, which it spends both on abstentions and on passing local patches above $\mathrm{RPS} = 1$; a gate is only meaningful where there is somewhere to escalate to, which is why it lives here rather than in the frozen configuration. **CoEP$_{\mathbf{L+SL}}$** is CoEP$_{\mathrm{L}}$ re-run with the rules self-learning accumulated, merged and frozen before evaluation so the arm is order-independent; the induction calls are priced to it rather than to CoEP$_{\mathrm{LF}}$, which never loads those rules. The submissions a rule was induced from are the only ones for which it is not a test of generalization, and are reported separately.

Corpus	Problems	Buggy	Correct	Tests	LOC
Refactory	5	1,783	2,442	7.4	11.9
Codeforces	6	1,187	698	35.5	12.3
Total	11	2,970	3,140	22.7	12.0

Table 1: Dataset statistics. Tests is the average test-suite size per problem; LOC is the average number of non-blank lines per submission.

Datasets. We evaluate on 11 problems and 2,970 buggy submissions drawn from two real-student corpora (Table 1). Refactory contributes five CS1-level assignments (Hu et al. 2019), the standard corpus of this literature (Ahmed et al. 2022; Zhang et al. 2024; Orvalho, Janota, and Manquinho 2025). Codeforces contributes six competitive-programming problems from the open-r1 release (Penedo et al. 2025), spanning difficulty ratings from 800 to 2000, which cover five official rating bands from Newbie to Candidate Master (Codeforces); we keep only problems whose complete official test suites are released, so the oracle is the judge's own. Every submission each corpus labels buggy is a repair target for every arm, so the denominator is the pool as published and is identical across arms. No train/test split is used, as prior per-assignment systems assume, and all results additionally hold with CoEP's build samples excluded from the denominator.

LLM Configuration. Every LLM arm runs on two models, one per provider: **GPT-5.4-mini** ($0.75/$4.50 per Mtok) and **Claude Sonnet 5** ($2/$10 per Mtok). We select each provider's cheapest tier at which prompt caching is functional, for two reasons. First, fairness to the caching baseline: PyDex$_{\mathrm{PC}}$ only competes if caching can engage, and the absolute cheapest tiers rule it out. We verified empirically that Claude Haiku 4.5 refuses to cache prompts below its 4,096-token minimum, above these repair prompts, and that GPT-5.4-nano's automatic caching did not engage even on identical 1,200-token prompts, whereas both selected models cache at their documented 1,024-token minimums. Second, cheap tiers keep every measured dollar gap a conservative floor, and the break-even N^* is price-invariant regardless (RQ3).

Evaluation Metrics. Metrics are grouped by the research question they serve.

RQ1 and RQ5 (repair effectiveness). Repair rate (RR): percentage of buggy submissions for which the system produces a program passing the full test suite. **Relative patch size (RPS)**: tree-edit distance between buggy and repaired ASTs normalized by buggy program size, reimplemented from Refactory's definition (Hu et al. 2019); smaller patches preserve the student's approach and are pedagogically preferable. RPS is never used to *claim* a repair the tests do not support; it enters the pipeline only in CoEP_{LF}, where a value above 1 routes a submission to the fallback and the smaller of the two patches is kept, so the gate can add a model call but cannot manufacture a repair or enlarge a patch. Repair rates carry 95% Wilson intervals. Comparisons between arms we ran ourselves are paired at the submission level and tested with exact McNemar, the exact form rather than the chi-square approximation because the discordant counts are often single-digit. That covers every pairing on Codeforces and every pairing against PyDex. It does not cover Refactory on its own corpus, where we quote its published per-assignment results and therefore have no per-submission outcomes to pair against; that one comparison is reported as unpaired rates with intervals, and we mark it as such wherever it appears.

RQ2 (cost). Average turnaround time (ATT): mean wall-clock seconds to repair one submission end to end, decomposed into ATT_{cpu} (local search on the CPU), ATT_{api} (model-call latency), and their sum ATT_{all}; CoEP's one-off build wall-clock is reported separately as $\text{ATT}_{\text{build}}$ per assignment, an intercept like C_{build} rather than a per-submission term. **API cost**: USD spent on model inference per repaired submission and per assignment (including a system's one-off build cost), priced at the tiers above from exact prompt and response token counts. Naming it for the API is deliberate: monetary inference spend is only one of the costs this paper argues about, and latency, determinism, and data egress are carried by ATT and the setup. The **abstention rate** f, the fraction of submissions the local tool escalates, is what the break-even $N^* = C_{\text{build}}/(\Delta(1-f))$ consumes. For RQ5 we additionally report the **gate rate**, the fraction of *repaired* submissions the threshold routes to the fallback, and sweep that threshold over the family of Section to show the resulting patch-size/spend frontier.

RQ3 and RQ4 (component contributions). The **repair-source split** attributes every fix to the mechanism that produced it (synthesized rule, generic rule, reference transplant, or LLM fallback). **Rule provenance** counts the rules the builder authored, the rules self-learning added, and the rules that actually fired at deployment, and tracks which recur across problems. For RQ4 the comparison is CoEP_{L} against $\text{CoEP}_{\text{L+SL}}$ on identical submissions, reported with and without the few each rule was induced from. **Diagnosis coverage** is the fraction of repaired submissions whose patch carries a misconception label (rule-fired repairs), quantifying how much of the instructor-facing profile the tool actually delivers rather than leaving it implicit.

Setup. All experiments run on a single Apple M1 Ultra workstation (64 GB RAM, macOS) under Python 3.13, CPU only for everything but the API calls. PyDex uses $K{=}10$ sampled candidates per submission, issued in parallel, with every call billed and timed. The three CoEP configurations are derived from a single pass over the submission stream with no duplicated LLM calls, so the reported local numbers are always the frozen tool's. Refactory runs with its original algorithm unchanged, ported to Python 3. We release the code, prompts, raw model outputs, and per-submission logs.

Results

We answer the five research questions stated in the Experimental Setup; together they cover the introduction's claims.

RQ1: Repair Effectiveness The central finding concerns CoEP's local repair against the strongest non-LLM tool. On the Refactory corpus of 1,783 buggy submissions, CoEP's local path of named rules, the shared generic library, and reference-guided transplant repairs **98.48%** of submissions at mean **0.289** RPS, against Refactory's own reported **90.80%** at **0.397**: higher coverage and a patch a quarter smaller, at zero marginal LLM cost. This comparison is unpaired, since Refactory's results on its own corpus are published per-assignment aggregates. Where we run Refactory ourselves, on Codeforces, the gap widens and the comparison is paired: 99.92% against 78.59%, exact McNemar $b{=}406$, $c{=}17$, $p{<}10^{-97}$. This is the result that makes a local tool worth synthesizing. It is not merely cheaper than a symbolic tool that consumes every correct solution; it repairs more.

Against the per-submission LLM baseline the separation is not in coverage. Over the whole benchmark CoEP_{L} reaches 99.06% and PyDex 98.96%, and the paired test finds no difference ($b{=}28$, $c{=}25$, $p{=}0.78$): on CS1-scale programs coverage is near-saturated for any competent repairer (Zhang et al. 2024). The separation is in *patch size*. CoEP_{L} edits 0.449 of a program against PyDex's 0.598, and adding the escalation tail takes CoEP_{LF} to **99.66%** at **0.398** — now significantly ahead of PyDex on coverage as well ($b{=}28$, $c{=}7$, $p{=}5.1{\times}10^{-4}$), with 95% Wilson intervals of $[99.38, 99.82]$ against $[98.52, 99.26]$. Because CoEP's patches are test-validated before a student sees them, its only local failure mode is abstention ($\perp$), never a wrong repair; abstentions take the per-submission LLM path, so the hybrid never sacrifices coverage. What this costs is RQ2.

Reading Table 2 by row rather than by column is what the per-assignment form is for. Refactory's rate swings from 40.3% to 100% across the eleven assignments while neither LLM arm drops below 93.8%, and the swing does not track difficulty: it repairs all three of the hardest problem's submissions (rating 2000) and four in ten of a rating-1600 problem's. What predicts its collapse is whether the correct pool holds a structurally matching reference, which RQ5 takes up. The rows also show where a low patch size is not a virtue. On Codeforces every system's patches inflate, and Refactory's 0.680 looks competitive against CoEP_{LF}'s 0.590 — but it is averaged over the 78.59% of submissions it accepts, where CoEP_{LF} repairs 100.00%. A tool measured only on the cases it accepts is not measured on the same problem. We report both columns rather than choosing the framing that flatters us.

	ID	Assignment	Rating	RR (%)				RPS			
				Refactory	PyDex	CoEP$_{\text{L}}$	CoEP$_{\text{LF}}$	Refactory	PyDex	CoEP$_{\text{L}}$	CoEP$_{\text{LF}}$
Refactory	R1	Sequential search	–	99.0	99.5	100.0	100.0	0.390	0.309	0.209	0.206
	R2	Unique dates/months	–	78.2	96.3	93.8	97.7	0.560	0.412	0.396	0.392
	R3	Duplicate elimination	–	97.4	100.0	100.0	100.0	0.340	0.381	0.276	0.276
	R4	Sorting tuples	–	88.2	99.4	100.0	100.0	0.310	0.521	0.337	0.241
	R5	Top-k elements	–	88.0	96.3	100.0	100.0	0.320	0.189	0.190	0.190
Codeforces	C1	Nearly Lucky Number	800	85.2	99.8	100.0	100.0	0.588	0.700	0.595	0.511
	C2	cAPS lOCK	1000	81.4	99.1	100.0	100.0	0.724	1.133	0.732	0.619
	C3	Planning The Expedition	1200	47.8	100.0	98.6	100.0	0.847	0.673	0.637	0.593
	C4	Measuring Lengths in Baden	1400	91.9	100.0	100.0	100.0	0.621	0.631	0.618	0.525
	C5	Jumping Jack	1600	40.3	100.0	100.0	100.0	0.878	0.959	0.932	0.823
	C6	Digital Root	2000	100.0	100.0	100.0	100.0	2.248	1.868	1.527	1.527
	Corpus (2,970 submissions)			**85.9**	**99.0**	**99.1**	**99.7**	**0.500**	**0.598**	**0.449**	**0.398**

Table 2: Per-assignment repair rate and mean relative patch size, Claude Sonnet 5 as builder, on 2,970 buggy submissions. RPS is averaged over the submissions each arm repairs, so a lower rate and a lower RPS in the same row mean the arm is measured on an easier subset. Every arm is given the same 2,970 inputs, including the 98 submissions that already pass; CoEP returns those unedited, at $\mathrm{RPS} = 0$, while both baselines rewrite them. PyDex$_{\text{PC}}$ is omitted: prompt caching changes price, not output, so its RR and RPS equal PyDex's to four decimals. Refactory's CS1 rows are its published per-assignment results; its Codeforces rows are our run of the released tool. Codeforces rows carry the problem's difficulty rating; the CS1 assignments are unrated. The ID column is how the rest of the paper refers to an assignment. The GPT-5.4-mini builder is reported in RQ5.

RQ1. CoEP's local repair reaches 98.48% RR at 0.289 mean RPS on the Refactory corpus, beating the classical non-LLM tool (90.80%, 0.397) on both coverage and patch size at zero marginal LLM cost. Against per-submission LLM repair, coverage is at parity (p=0.78) and patch size is a third smaller; escalation adds 99.66% coverage, ahead of PyDex at $p<10^{-3}$.

RQ2: Cost Effectiveness and Break-Even The systems differ in *cost structure*, not just constant. A per-submission LLM arm pays an inference on every submission, so its cost grows linearly with enrollment and recurs every term. CoEP pays a one-off synthesis cost C_{build} per assignment, then repairs the covered fraction at *zero* marginal LLM cost locally and pays a per-submission inference only on the escalating tail. Over the whole benchmark that is \$13.58 against PyDex's \$292.69 at the frontier tier, a factor of 22, and \$1.00 against \$28.42 at the cheap tier, a factor of 28. Of CoEP's total, \$12.22 is the one-off build and \$1.36 the entire deployment spend across 2,970 submissions.

Fig. 2(b) turns that into the quantity a department actually decides on. With Δ the marginal per-submission saving and f the escalation rate, synthesis wins past $N^* = C_{\text{build}}/(\Delta(1-f))$, which our measured build cost and escalation tail put at **11 submissions per assignment** against PyDex and 13 against PyDex$_{\text{PC}}$. Ten of our eleven assignments exceed that, most by an order of magnitude; the rug on the axis marks their actual sizes. Below N^* a per-query repairer is the right tool, and the paper claims nothing else.

Caching and cheaper tiers do not close it. The standard remedies for per-submission cost are provider prompt caching and cheaper model tiers, and the design measures both. PyDex$_{\text{PC}}$ re-prices every PyDex query under the provider's documented caching rules, as an upper bound that assumes a cache hit wherever one is allowed; every LLM arm already runs at the cheapest tier at which caching functions. The discount is 10.2% (\$292.69 to \$262.97), and the measurement exposes why it is small: caching pays for a shared leading *prefix*, but the shared material in these prompts is not a prefix. The variable buggy program and retrieved exemplars come first, leaving an identical prefix of about ten tokens; what remains cacheable is the reuse within one submission across its K sampled candidates, and that is what PyDex$_{\text{PC}}$ realizes. The break-even bound makes the same point analytically: N^* is invariant to token price under the stated assumption that build and per-query tokens are billed on the same schedule, since C_{build} and Δ scale together. The claim is about how many model calls per specification are made, not the price of a call — and because every arm runs at the cheapest tier, the measured dollar gaps are a *floor* on the effect rather than its size.

Turnaround is a trade, not a sweep. At the frontier tier CoEP$_{\text{LF}}$ answers in 3.06 s against PyDex's 14.64 s, and the split is the whole story: CoEP spends 2.69 s on local CPU search and 1.36-worth of API on a tenth of submissions, while PyDex spends 0.66 s locally and 13.97 s waiting on the network. At the cheap tier the ordering reverses — PyDex answers in 2.14 s against our 3.22 s — because a small model returns a single completion faster than our search enumerates and re-executes candidates. We report it rather than quoting only the tier that flatters us. The trade is what the two seconds buy: the local answer is deterministic, has no rate limit, and never leaves the institution, and it is unaffected by the deadline-night spikes that make a hosted queue slowest exactly when a class needs it most.

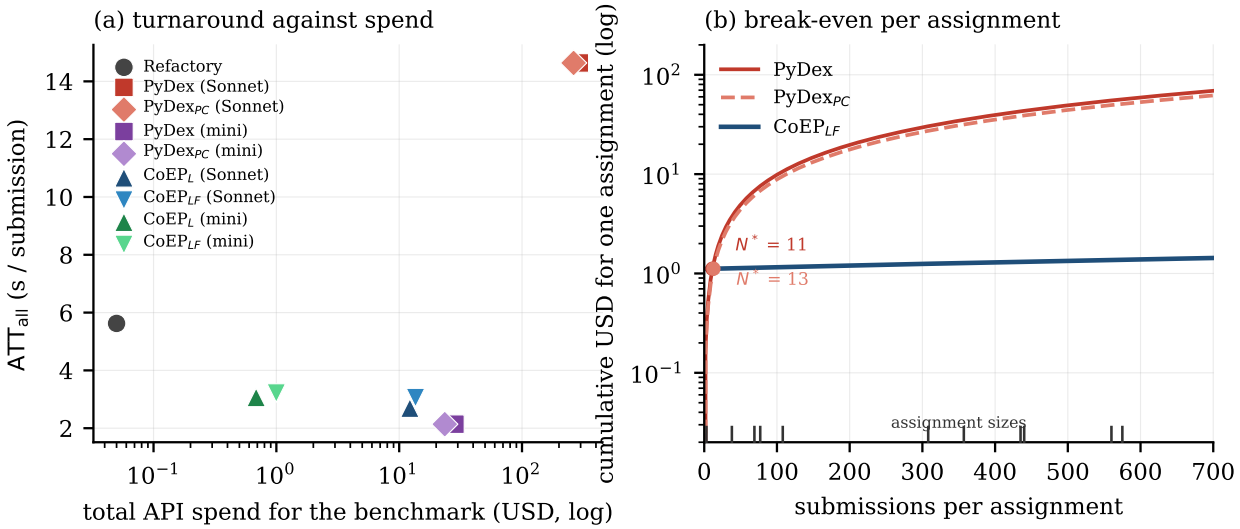


Figure 2: Cost, and where amortization overtakes per-submission inference. Total spend is for all 2,970 submissions and includes CoEP's one-off build; the local search runs on a CPU, so its turnaround carries no network dependency, rate limit, or nondeterminism.

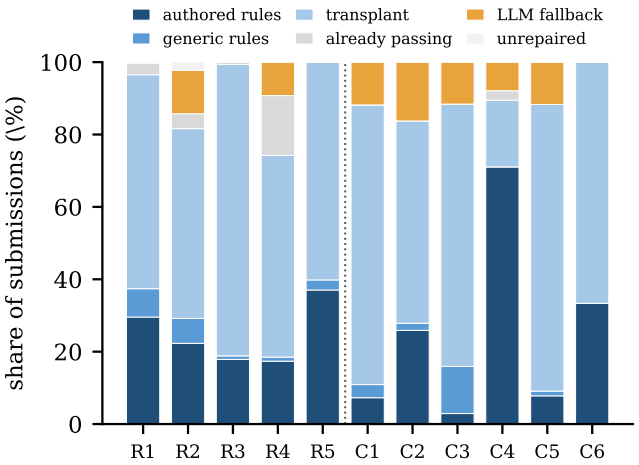


Figure 3: What produced each repair, assignment by assignment (IDs as in Table 2; the dotted rule separates the CS1 corpus from Codeforces).

Repair source	share (%)	mean RPS	labelled
LLM-authored rules	21.8	0.124	•
Generic-library rules	4.1	0.025	•
Reference transplant	70.8	0.595	–
Already passing (no edit)	3.3	0.000	–
LLM fallback (CoEP$_{LF}$)	8.5	0.775	–
Framework-only ablation		*[pass not yet run]*	

Table 3: Repair source, patch size, and whether the patch carries a misconception label. The final rows are the *framework-only* ablation: the identical tool with the LLM-authored rules removed.

RQ2. CoEP replaces a per-submission inference with a one-off \$12.22 synthesis plus a \$1.36 escalation tail, 22× cheaper than per-submission repair over 2,970 submissions and breaking even at N^*=11 per assignment. Caching recovers only 10.2% and N^* is price-invariant, so the gap is structural rather than a matter of tier.

RQ3: Where the Repairs Come From Two complementary local mechanisms. CoEP's local repair combines named rules (problem-specific plus the shared generic library) with reference-guided transplant, and the two carry different loads. The rules make small, labelled edits for the common misconceptions — a relaxed comparison, an off-by-one bound — and they are surgical: 25.9% of repaired submissions at mean RPS 0.108, with the always-on generic library alone at 0.025. Most student bugs are not single-token slips, though; they need a medium, structural edit no token-level rule expresses, and the transplant supplies it by adopting the smallest set of differing hunks from a structurally-matched correct program, variable-renamed to fit. It carries 70.8% of repairs at 0.595, five and a half times more of the student's program disturbed. The transplant is what lifts a rule-only tool to competitive coverage; the rules keep the common cases at near-minimal patch size and are the only layer that leaves a diagnosis behind.

Fig. 3 shows the division is not uniform. Where the correct pool is deep and the assignment small, the transplant dominates and the authored rules take a fifth of the work; where one misconception dominates a class the authored layer takes it over: on C4 a single rule, `carry_fix_two_value_print`, repairs 27 of the assignment's 38 submissions, and the authored layer accounts for 78.9% of them. That is the shape the once-per-assignment

build is meant to exploit: the LLM's contribution is largest exactly where the class shares one mistake.

What the LLM layer buys. A *framework-only* ablation, the identical tool with the LLM-authored rules removed and the generic library and transplant left in place, isolates it. *[Ablation cells from the running pass.]* The provenance logs say what to expect and why. The builder authored 91 rules that fired at all, against the generic library's 5, and they fired 789 times against the library's 293 — but only one authored rule fired on more than one assignment, while `generic_flip_compare` alone fired 198 times across 8 of the 11. The authored layer is assignment-specific by construction, which is the point: it is bought once per assignment and encodes what *this* class got wrong.

Escalation as a fourth source. CoEP_{LF} adds one more producer. The threshold fires on 317 submissions, 10.8% of those repaired locally, and the model returns the smaller patch on 232 of them (73.2%), for $1.36 across the benchmark. Those are the cases where no reference aligned hunk-wise and the transplant fell back on rewriting; the escalation is what stops a rewrite from being reported as a repair.

Diagnosis for free. Every rule-produced patch carries the ordered rule names that produced it, so aggregated over a class the tool yields a misconception profile: how often each named mistake occurred, and on which programs. The names are legible as misconceptions rather than as patches — `append_missing_return_len_after_loop`, `count_compare_relax`, `slice_case_check_to_equality` — and an instructor gets the report at no extra cost, from an artifact that can be read and edited. Diagnosis coverage, the fraction of local repairs carrying such a label, is 25.9% (Table 3); a per-submission API emits none of it at any price.

RQ3. Named rules produce 25.9% of local repairs at mean RPS 0.108, five times smaller than the transplant's 0.595 and each carrying a misconception label; the transplant produces 70.8% and is what carries coverage. Authored rules are assignment-specific (one of 91 recurs across problems) while the generic library recurs everywhere, which is why the build is priced per assignment.

RQ4: Learning From Its Own Escalations $\text{CoEP}_{\text{L+SL}}$ is the frozen tool re-run with the rules absorbed from its own fallback fixes. Absorption happens outside the evaluated pass and the augmented tool is frozen before this arm runs, so the comparison against CoEP_{L} is order-independent: the same submissions, the same acceptance rule, two rule sets. Absorption learns only from submissions the local tool fails outright, never from threshold escalations, so its supply of material is exactly the abstention set.

The first result is how little there is to learn from. Across the 22 (assignment, builder) cells the frozen tool abstains on 28 submissions in total, concentrated on two assignments, and 19 of the 22 cells leave nothing to absorb at all. That is not a failure of the mechanism; it is what a 99.06% local repair rate means. Where the tool does abstain, absorption fires: four rules on R2 with the frontier builder, three on R2 with the cheap one, and one on C3.

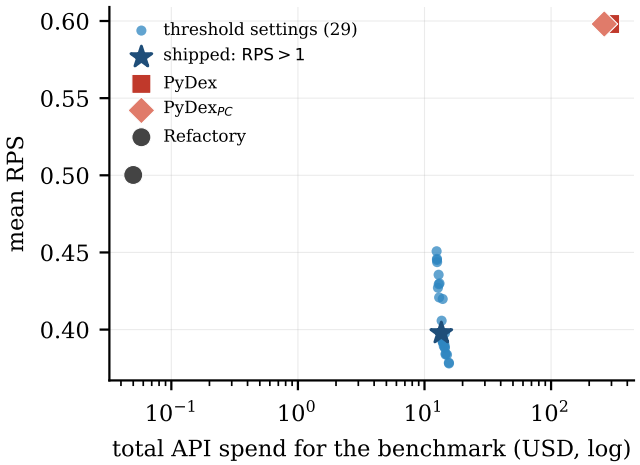


Figure 4: Every threshold setting, against the baselines it is compared to.

Where it fires, the rules transfer. On R2 the four absorbed rules take the frozen tool from 93.79% to **95.40%**, and the cheap builder's three take it from 93.79% to **96.55%**; on C3 a single rule, `binarysearch_zero_exit`, closes the last gap to 100%. Paired at the submission level the gain is significant for both builders — exact McNemar b=9, c=1, p=0.021 at the frontier tier and b=12, c=0, p=4.9×10^{-4} at the cheap one. The cheap builder learning *more* than the frontier one is worth stating: the material is the same 27 abstentions either way, and what differs is how general a rule the model writes from one fix.

Eight submissions induced these rules out of the 504 evaluated across the three cells, so excluding them moves the rates by under two points and the gain is measured almost entirely on programs the rules were never validated against. Induction cost $0.76 in total, priced to the artifact rather than to the configuration it was gathered alongside. The learned names read as misconceptions rather than as patches — `uniqueness_count_gt1_to_neq1`, `counter_var_confusion`, `index_vs_count_and_uncalled_bool` — which is what makes them worth carrying into the next term.

RQ4. Absorption fires on 3 of 22 cells, because on the other 19 the frozen tool leaves no miss to learn from. Where it fires the learned rules generalize: R2 moves from 93.79% to 95.40% (frontier) and 96.55% (cheap), C3 to 100%, for $0.76 of induction, significant at p=0.021 and $p<10^{-3}$ on submissions the rules were not induced from.

RQ5: What the Conclusion Survives **Difficulty.** The six Codeforces problems climb from rating 800 to 2000, and the deep solution pools of the CS1 assignments give way to sparser, more diverse ones as difficulty rises. The symbolic baseline tracks that: 90.8% on CS1, 83.1% on the 800–1000 band, and **54.3%** on 1200–2000 (Table 4). CoEP_{LF} holds 99.4%, 100.0%, 100.0%. But the per-assignment rows say the driver is not difficulty. Refactory repairs all three submissions of the hardest problem (C6, rating 2000) and four in ten of

Stratum	Refactory	PyDex RR (%) / RPS	CoEP_{LF}
Difficulty (Sonnet builder)			
CS1 (Refactory)	90.8 / 0.397	98.6 / 0.382	99.4 / 0.269
CF 800–1000	83.1 / 0.663	99.4 / 0.942	100.0 / 0.572
CF 1200–2000	54.3 / 0.822	100.0 / 0.802	100.0 / 0.689
Builder			
Claude Sonnet 5	–	99.0 / 0.598	99.7 / 0.398
GPT-5.4-mini	–	98.9 / 0.670	99.7 / 0.411
Threshold, 29 measured settings			
range	–	–	99.7 / 0.378–0.451
spend		\$12.47–\$15.61 total	

Table 4: The three axes the conclusion is asked to survive: assignment difficulty, LLM provider, and the escalation threshold.

C5 (rating 1600); its rate does not decrease monotonically in rating at all. What predicts the collapse is whether the correct pool holds a structurally matching reference, since block-level reuse starves when solution strategies diverge — and C5's 77 submissions draw on a pool of two correct solutions. Patch size tells the same story from the other side: every arm's RPS roughly doubles from CS1 to the hard band, so the ordering between arms survives while the absolute numbers do not.

Provider. Every arm runs identically on GPT-5.4-mini and Claude Sonnet 5, so a conclusion that held for one provider only would surface here. It does not: CoEP_{LF} reaches 99.7% at 0.398 RPS with the frontier builder and 99.7% at 0.411 with the cheap one, a 3% difference in patch size for an 18-fold difference in build price (\$12.22 against \$0.68). The ordering against both baselines is identical under either builder, and RQ4 showed the cheap builder can learn the *better* rule. What the frontier model buys here is a slightly smaller patch, not a different conclusion.

Threshold. The local tool accepts whatever passes the tests, so its patch-size distribution has a tail the acceptance rule does nothing about: the largest local patches exceed the size of the programs they repair. Escalating that tail is what CoEP_{LF} spends on, and the question is how much the choice of where to cut matters. We swept 38 candidate rules — median- or mean-anchored with MAD, standard-deviation or interquartile spread, bare quantiles, multiples of the assignment's median, corpus-wide variants of each, and fixed constants — of which 29 give distinct settings that stay inside the cached escalation prefix and are therefore measured exactly. Three things hold across all of them (Fig. 4). Repair rate is *identical* to four decimals at 99.66%, because the threshold only re-examines patches the tests already passed and the smaller of the two is kept: the gate can cost a call but cannot lose a repair. Mean RPS falls monotonically with the escalation rate, from 0.451 to 0.378, with no interior optimum — so the setting is a budget choice rather than a discoverable value. And total spend across the entire family stays within \$12.47–\$15.61, a band narrower than the width of a marker beside PyDex's \$292.69. Every one of the 29 settings beats all three baselines on repair rate, patch size, turnaround and dollars at once.

Two findings inside the sweep are worth stating because they are negative. Making the threshold *relative to the assignment* does not pay: a single corpus-wide cutoff sits on the same frontier or better, since RPS is already normalized by program size and a second per-assignment normalization directs calls toward assignments where the tool is already doing well. And the median-plus-MAD family we began with is Pareto-dominated inside the sweep — it buys the same patch-size reduction for more calls. The rule we ship is therefore the one whose value is a definition rather than a fitted constant. Its honest cost is that it is nearly inert where local patches are uniformly small: it fires on 0.0% of R3, 0.7% of R1 and 0.9% of R5, against 22.3% of C2. A relative rule would wake on those three; it would also cost more everywhere else, and Fig. 4 shows the whole argument survives either way.

RQ5. Yes on all three axes. The symbolic baseline falls from 90.8% to 54.3% between the CS1 corpus and the hardest Codeforces band while CoEP_{LF} holds 100%; the two builders agree to 3% in patch size for an 18-fold difference in build price; and across 29 measured escalation thresholds the repair rate is invariant, patch size spans 0.378–0.451 and total spend \$12.47–\$15.61, so every setting beats every baseline on all four metrics.

Related Work

APR for programming assignments. The pre-LLM systems surveyed in the introduction (Yi et al. 2017; Gulwani, Radicek, and Zuleger 2018; Wang, Singh, and Su 2018; Hu et al. 2019; Ahmed et al. 2022) exploit per-assignment structure, and the reference-reuse ones among them depend on finding a structurally matching correct solution; our transplant stage stands openly in their reference-reuse lineage, at hunk rather than block granularity, and our head-to-head run shows the composed artifact exceeding the strongest of them on both coverage and patch size. Closest to our synthesis layer, Refazer learns per-assignment syntactic transformations from (buggy, fixed) student pairs with PROSE (Rolim et al. 2017); CoEP replaces the example-driven synthesizer with a cheapest-tier LLM that *names* the misconception behind each rule, and surrounds the learned layer with generic search, reference transplant, and harness-gated escalation. At AAAI the setting stays active from the per-submission side: counterexample-guided and preference-tuned minimal repair at AAAI-25 (Orvalho, Janota, and Manquinho 2025; Dai et al. 2025), and AAAI-26's learner-tailored repair, which retrieves per-assignment example fixes to guide each query (Dai et al. 2026), amortizing *data* into a retrieval index while every repair still pays for an LLM call (our PyDex arm bounds this design's economics). CoEP instead removes the per-submission call for everything its artifact covers and pays only on escalation.

LLM-based repair. AlphaRepair recast repair as generation (Xia and Zhang 2022); ChatRepair adds conversational test feedback and per-bug cost accounting (Xia and Zhang 2024); PyDex/MMAPR target student programs (Zhang et al. 2024, 2022); FastFixer accelerates per-query inference (Liu

et al. 2024); CREF adds a tutor in the loop (Yang et al. 2024). All are per-submission; CoEP is complementary (any can serve as its fallback) but shifts the primary cost to the assignment.

Templates, cost-efficient inference, and tool synthesis. PAR and TBar showed a fixed library of human-curated fix templates repairs a surprising share of real bugs (Kim et al. 2013; Liu et al. 2019); CoEP's generic layer is a mutation-inversion instance (Just et al. 2014), and its novelty is the *synthesized, assignment-specific* template layer, authored per specification by an LLM. FrugalGPT cascades, SRepair's cheap pipelines, and provider caching (Chen, Zaharia, and Zou 2024; Xiang et al. 2024; Anthropic 2025; OpenAI 2024) all optimize *within* the pay-per-query paradigm and compose with CoEP, which instead removes queries. Voyager and LLMs-as-tool-makers amortize agent-written code across episodes (Wang et al. 2024; Cai et al. 2024), self-refinement improving one-shot generations (Madaan et al. 2023; Shinn et al. 2023); CoEP adds what education forces: verifiability (never-trust-self-reports) and minimal, measured patches, because its output is feedback to a learner.

Ethical Statement

CoEP automates feedback, not grading, under instructor oversight; unhandled students get full-LLM treatment; repairs are auditable; data are public.

References

Ahmed, U. Z.; Fan, Z.; Yi, J.; Al-Bataineh, O. I.; and Roychoudhury, A. 2022. Verifix: Verified Repair of Programming Assignments. *ACM Transactions on Software Engineering and Methodology*, 31(4).

Anthropic. 2025. Prompt Caching — Claude API Documentation. https://docs.anthropic.com/en/docs/build-with-claude/prompt-caching. Cache reads are billed at 0.1x the base input-token price; accessed July 2026.

Cai, T.; Wang, X.; Ma, T.; Chen, X.; and Zhou, D. 2024. Large Language Models as Tool Makers. In *The Twelfth International Conference on Learning Representations, ICLR 2024*.

Chen, L.; Zaharia, M.; and Zou, J. 2024. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. *Transactions on Machine Learning Research*.

Codeforces. ???? Codeforces Rating System. https://codeforces.com/blog/entry/68288. Rating bands: Newbie, Pupil, Apprentice, Specialist, Expert, ...

Dai, Z.; Chen, B.; Zhao, Z.; Tang, X.; Wu, S.; Yao, C.; Gao, Z.; and Chen, J. 2025. Less is More: Adaptive Program Repair with Bug Localization and Preference Learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Dai, Z.; Zhao, Z.; Wang, H.; Tang, X.; Wu, S.; Yao, C.; Gao, Z.; and Chen, J. 2026. Learner-Tailored Program Repair: A Solution Generator with Iterative Edit-Driven Retrieval Enhancement. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 30467–30475.

Gulwani, S.; Radicek, I.; and Zuleger, F. 2018. Automated Clustering and Program Repair for Introductory Programming Assignments. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2018*, 465–480. ACM.

Hu, Y.; Ahmed, U. Z.; Mechtaev, S.; Leong, B.; and Roychoudhury, A. 2019. Re-Factoring Based Program Repair Applied to Programming Assignments. In *34th IEEE/ACM International Conference on Automated Software Engineering, ASE 2019*. IEEE.

Just, R.; Jalali, D.; Inozemtseva, L.; Ernst, M. D.; Holmes, R.; and Fraser, G. 2014. Are Mutants a Valid Substitute for Real Faults in Software Testing? In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering, FSE 2014*. ACM.

Kim, D.; Nam, J.; Song, J.; and Kim, S. 2013. Automatic Patch Generation Learned from Human-Written Patches. In *Proceedings of the 35th International Conference on Software Engineering, ICSE 2013*. IEEE.

Liu, F.; Liu, Z.; Zhao, Q.; Jiang, J.; Zhang, L.; Li, G.; Sun, Z.; Li, Z.; and Ma, Y. 2024. FastFixer: An Efficient and Effective Approach for Repairing Programming Assignments. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, ASE 2024*. ACM.

Liu, K.; Koyuncu, A.; Kim, D.; and Bissyandé, T. F. 2019. TBar: Revisiting Template-Based Automated Program Repair. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2019*, 31–42. ACM.

Madaan, A.; Tandon, N.; Gupta, P.; Hallinan, S.; Gao, L.; Wiegreffe, S.; Alon, U.; Dziri, N.; Prabhumoye, S.; Yang, Y.; Gupta, S.; Majumder, B. P.; Hermann, K.; Welleck, S.; Yazdanbakhsh, A.; and Clark, P. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems 36, NeurIPS 2023*.

OpenAI. 2024. Prompt Caching — OpenAI API Documentation. https://platform.openai.com/docs/guides/prompt-caching. Automatic prefix caching with discounted cached input tokens; see also https://openai.com/index/api-prompt-caching/; accessed July 2026.

Orvalho, P.; Janota, M.; and Manquinho, V. 2025. Counterexample Guided Program Repair Using Zero-Shot Learning and MaxSAT-based Fault Localization. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Penedo, G.; Kydlíček, H.; Allal, L. B.; Lozhkov, A.; Bakouch, E.; von Werra, L.; and Wolf, T. 2025. Codeforces. https://huggingface.co/datasets/open-r1/codeforces.

Rolim, R.; Soares, G.; D'Antoni, L.; Polozov, O.; Gulwani, S.; Gheyi, R.; Suzuki, R.; and Hartmann, B. 2017. Learning Syntactic Program Transformations from Examples. In *Proceedings of the 39th International Conference on Software Engineering (ICSE)*, 404–415.

Shinn, N.; Cassano, F.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems 36, NeurIPS 2023*.

Wang, G.; Xie, Y.; Jiang, Y.; Mandlekar, A.; Xiao, C.; Zhu, Y.; Fan, L.; and Anandkumar, A. 2024. Voyager: An Open-Ended Embodied Agent with Large Language Models. *Transactions on Machine Learning Research*. ArXiv:2305.16291.

Wang, K.; Singh, R.; and Su, Z. 2018. Search, Align, and Repair: Data-Driven Feedback Generation for Introductory Programming Exercises. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2018*. ACM.

Xia, C. S.; and Zhang, L. 2022. Less Training, More Repairing Please: Revisiting Automated Program Repair via Zero-Shot Learning. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2022*. ACM.

Xia, C. S.; and Zhang, L. 2024. Automated Program Repair via Conversation: Fixing 162 out of 337 Bugs for $0.42 Each using ChatGPT. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2024*, 819–831. ACM.

Xiang, J.; Xu, X.; Kong, F.; Wu, M.; Zhang, Z.; Zhang, H.; and Zhang, Y. 2024. How Far Can We Go with Practical Function-Level Program Repair? arXiv preprint arXiv:2404.12833.

Yang, B.; Tian, H.; Pian, W.; Yu, H.; Wang, H.; Klein, J.; Bissyandé, T. F.; and Jin, S. 2024. CREF: An LLM-Based Conversational Software Repair Framework for Programming Tutors. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2024*, 882–894. ACM.

Yi, J.; Ahmed, U. Z.; Karkare, A.; Tan, S. H.; and Roychoudhury, A. 2017. A Feasibility Study of Using Automated Program Repair for Introductory Programming Assignments. In *Proceedings of the 11th Joint Meeting on Foundations of Software Engineering, ESEC/FSE 2017*, 740–751. ACM.

Zhang, J.; Cambronero, J.; Gulwani, S.; Le, V.; Piskac, R.; Soares, G.; and Verbruggen, G. 2022. Repairing Bugs in Python Assignments Using Large Language Models. arXiv preprint arXiv:2209.14876. Preprint; the archival version of this work appeared as PyDex at OOPSLA 2024 (Zhang et al. 2024).

Zhang, J.; Cambronero, J. P.; Gulwani, S.; Le, V.; Piskac, R.; Soares, G.; and Verbruggen, G. 2024. PyDex: Repairing Bugs in Introductory Python Assignments using LLMs. *Proceedings of the ACM on Programming Languages*, 8(OOPSLA1).

Reproducibility Checklist

General paper structure. Conceptual outline of the approach: yes (Figure 1 and the Approach section's stage specification). Claims vs. speculation delineated: yes (RQ answer boxes state measured claims; the break-even analysis in RQ2–RQ3 is marked as derivation over measured quantities). Pedagogical references for background: yes (Related Work).

Theoretical contributions. This paper makes no formal theoretical claims; the break-even expression in RQ2 is elementary algebra over measured quantities, with all symbols defined in the text.

Datasets. All experiments use two publicly available corpora: the Refactory dataset (Hu et al. 2019) and Codeforces problems and submissions from the open-r1 release (Penedo et al. 2025). Our scripted import pipeline is released, covering test parsing, per-test entry points, and the oracle-fidelity audit, and we state the problem-selection rule we applied by hand, so the exact evaluation sets are reconstructible from the public corpora. No new human-subject data are collected.

Computational experiments. Source code, prompts, synthesized repairers, raw LLM responses, the ported Refactory-tool configuration, and analysis scripts are released; experiments run on CPU only. Model versions, pricing snapshot, tokenizer convention, decoding budget (single sample per call), execution timeouts, and the no-split whole-pool protocol are stated in Experimental Setup and fixed in the released configuration. Metrics are formally defined; repair rates carry 95% Wilson intervals and paired claims use exact McNemar tests. Local search and evaluation are deterministic. Builder synthesis is stochastic; every round's prompt, response, and build-sample outcome for both builder tiers is released with the artifacts.